バージョン管理とは

慶應義塾大学理工学部物理情報工学科
渡辺

バージョンを管理するということ

プログラム開発

実行

開発

デバッグ

論文執筆

添削

執筆

多くの知的生産活動は、修正を繰り返す

バージョンを管理するということ

どっちが最新？

仕様書_佐藤修正_吉本追記.docx

仕様書_最終版_田中修正v2.docx

バージョン管理システム
(Version Control System, VCS)

バージョン管理システム

バージョン管理システムとは？

- ファイルの編集履歴を管理するためのシステム
- 編集履歴をすべて保存する「リポジトリ」というデータベースを持つ
- ユーザはリポジトリにアクセスしながら開発を行う

何ができるようになるの？

- 任意の時点に状態を戻すことができる
- 任意の時点間の差分を確認できる
- 誰が、いつ、どこを修正したか確認できる

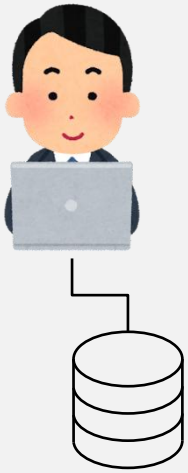
超優秀な秘書のようなもの

バージョン管理システムの歴史

歴史的に大きく分けると、以下の三種類

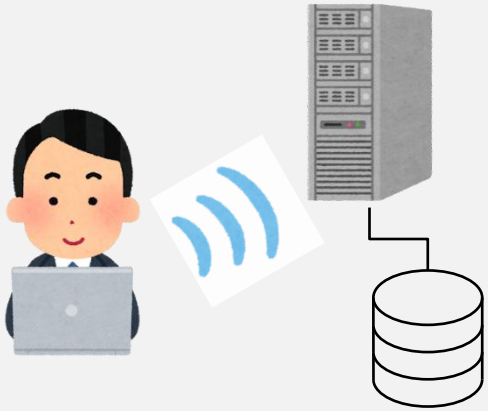
ローカル型

SCCS, RCS等



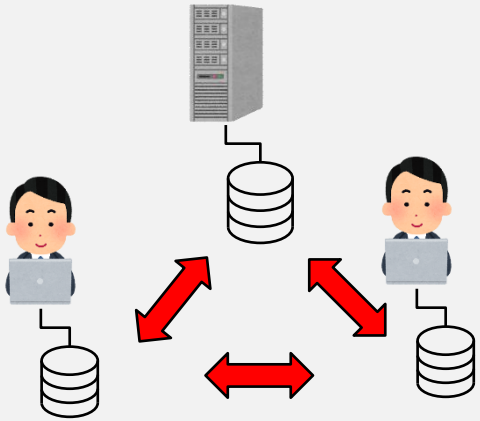
クライアント・サーバ型

CVS, Subversion等



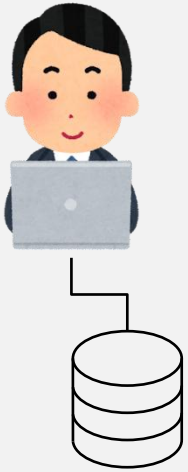
分散型

Mercurial, Git等



バージョン管理システムの歴史

第一期：ローカル型



1992年 Source Code Control System (SCCS)

- (おそらく)世界初のVCS
- IBM System/370向けに開発、後にPDP-11へ移植
- 複数のバージョンを一つのファイルに保存

1982年 Revision Control System (RCS)

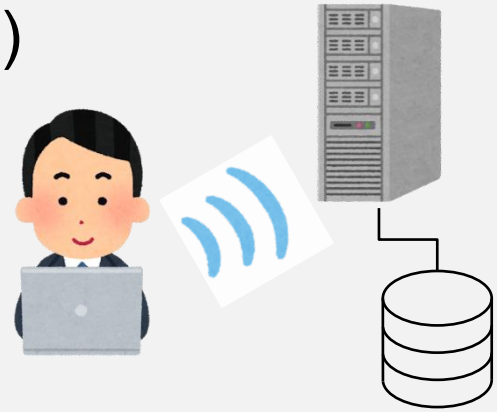
- バージョン管理はファイル単位
- ファイルを修正する時に「チェックアウト」する

ファイル修正時にロックをかける(排他制御)
→ 複数の人が同時に編集できない
管理がファイル単位
→プロジェクトとして管理できない

バージョン管理システムの歴史

第二期：クライアント・サーバ型

1986年 Concurrent Version System (CVS)



- RCSのフロントエンド
- リモートリポジトリを導入
- 複数の人が同時に修正可能
- 「マージ」の導入

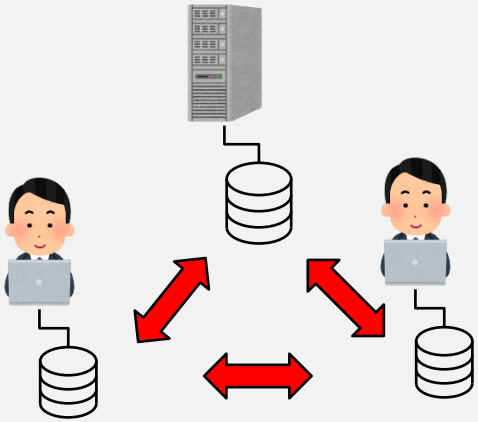
2000年 Subversion

- ファイル名変更やディレクトリの管理をサポート
- プロジェクト全体にバージョン番号(リビジョン)を付与

中央集権的なリモートリポジトリ
→ 全ての歴史がリモート側にある
→ 単一障害点になってしまう

バージョン管理システムの歴史

第三期：分散型



2000年 BitKeeper

- ローカルに全ての情報がある(分散型)
- Linuxカーネル開発に使われた商用ソフトウェア

2005年 Git

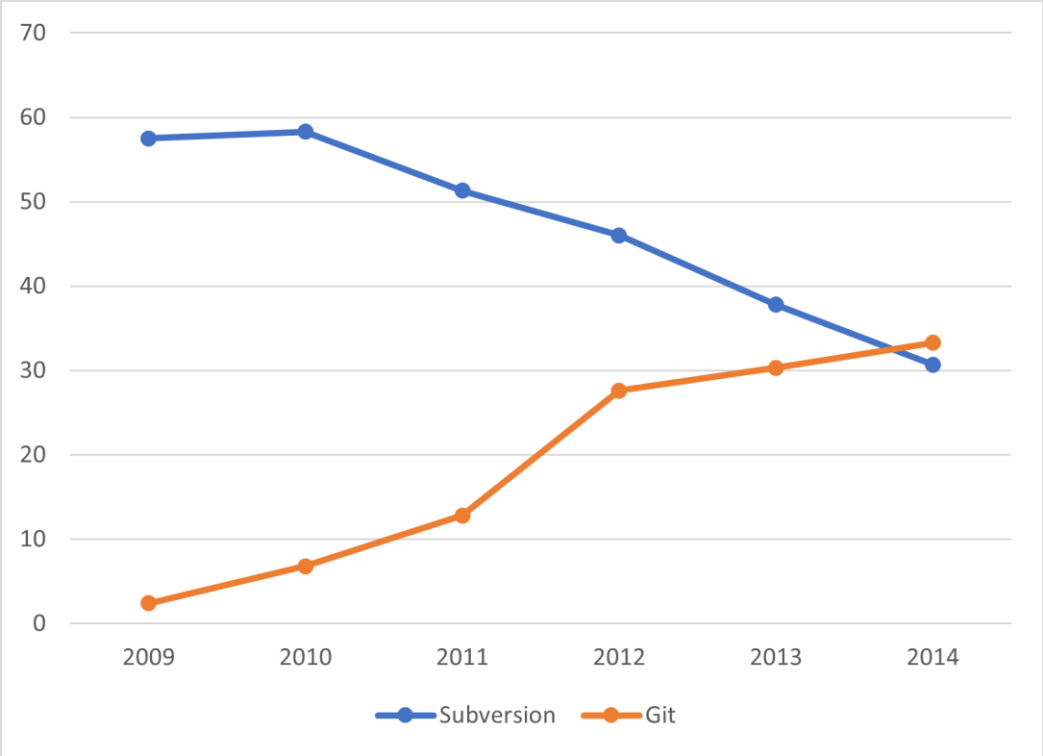
- 一部のLinux開発者がBitKeeperをリバースエンジニアリング
- BitKeeperを使えなくなったLinusがGitを開発
- 高速なブランチ切り替えやマージ

全てのリポジトリが完全な履歴を持つ
リモートとローカルの歴史の整合性を取る

バージョン管理システムの歴史

SubversionとGitのシェア

利用率(%)



Eclipse Community Survey

2018年のStackoverflow Surveyでは、Git 87.2%、Subversion 16.1%

現在、VCSとしてはGitが大きなシェアを持っている

GitとGitHub

Git

バージョン管理システム
履歴などを管理する
自分にとって「ローカル」

Git logo by Jason Long CC BY 3.0

GitHub

Gitのホスティングサービス
データや履歴をリモートにも保存
ソフトウェア開発のプラットフォーム
自分にとって「リモート」

※ Gitのホスティングサービスとしてはは GitLabも有名

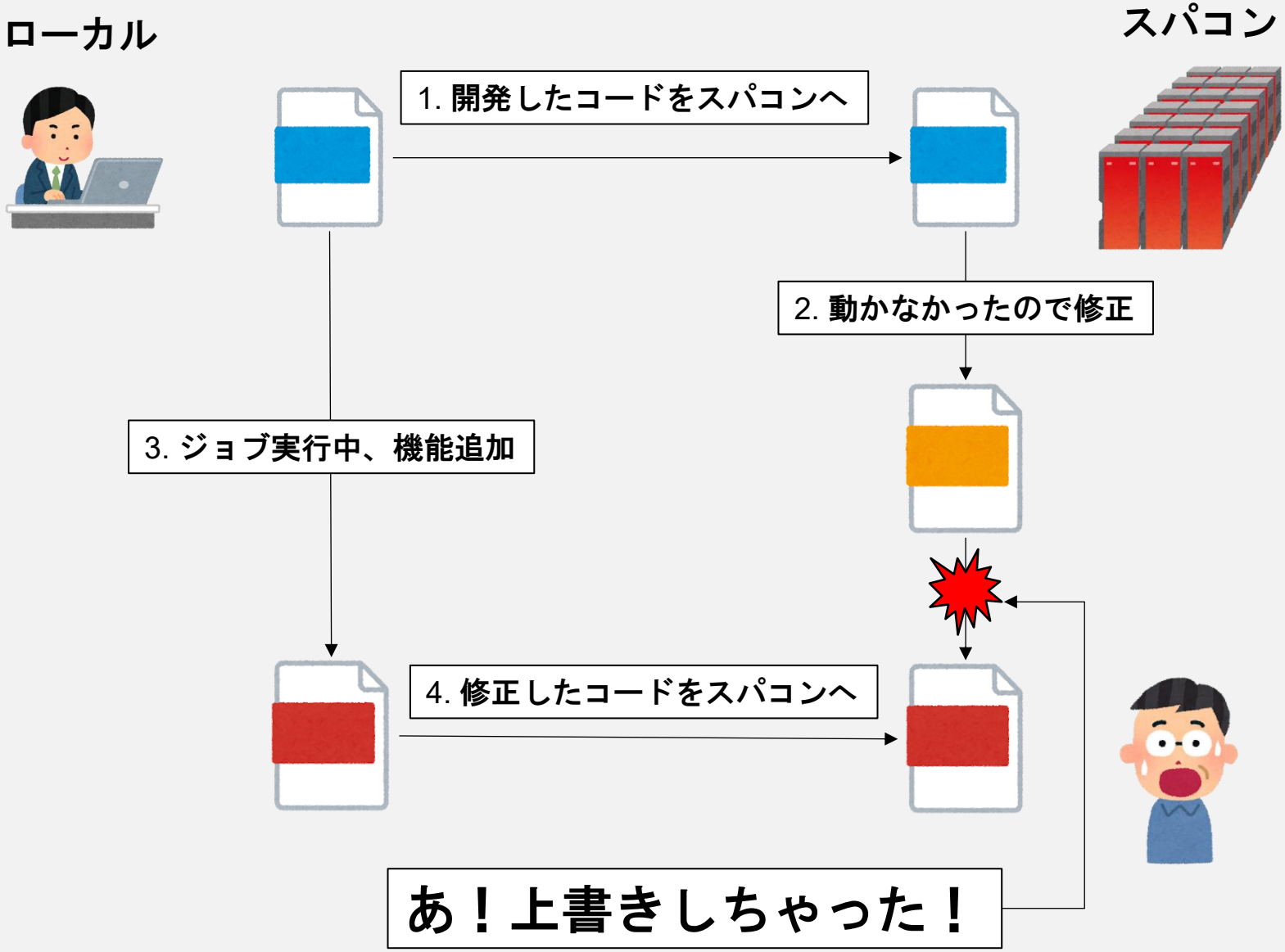
ありがちなパターン1

1. 自宅と大学で論文執筆作業

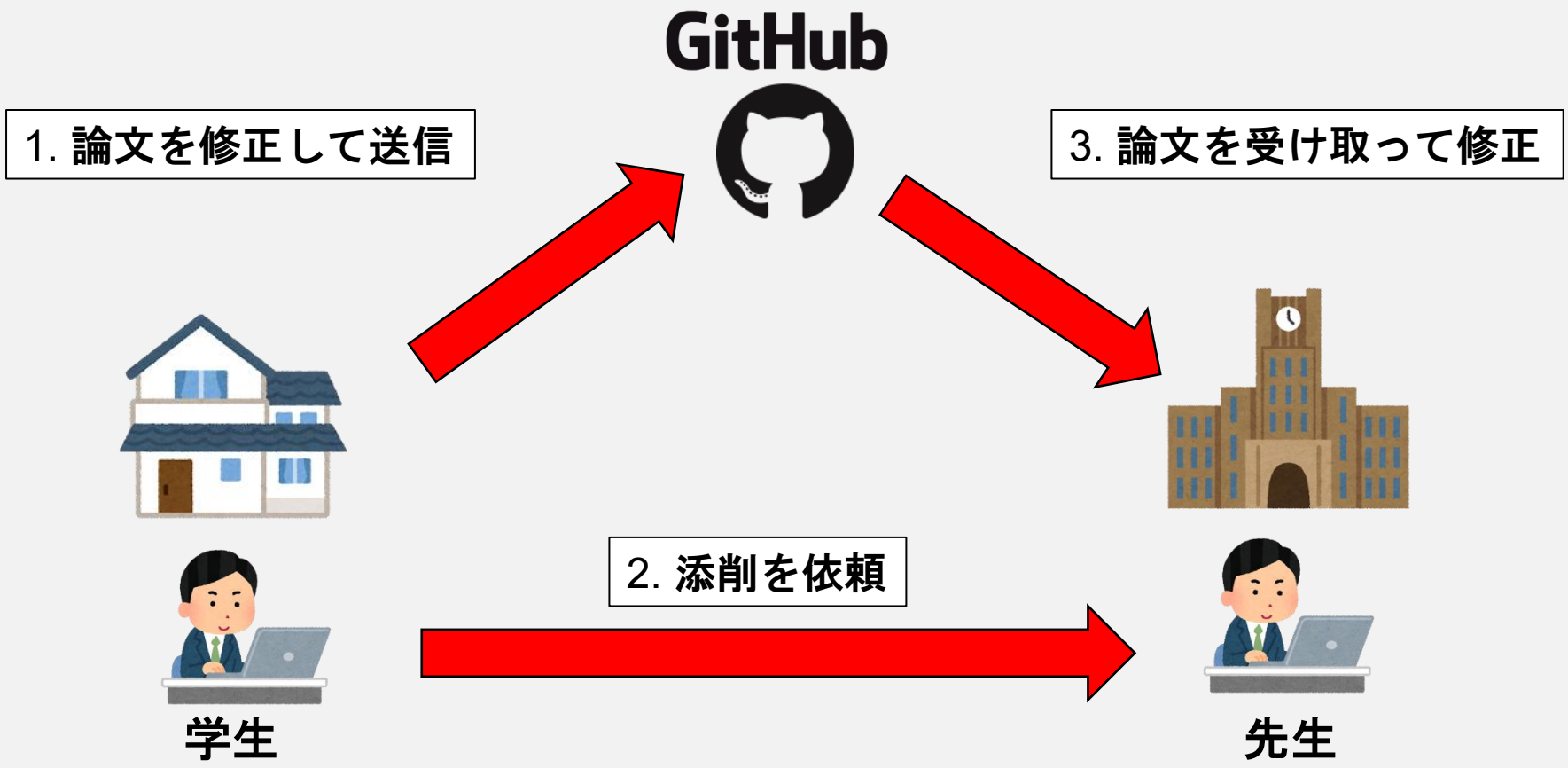
2. 先生に論文添削を依頼

3. 添削が返ってきた時、古いバージョンを渡してしまったことに気づく

ありがちなパターン2

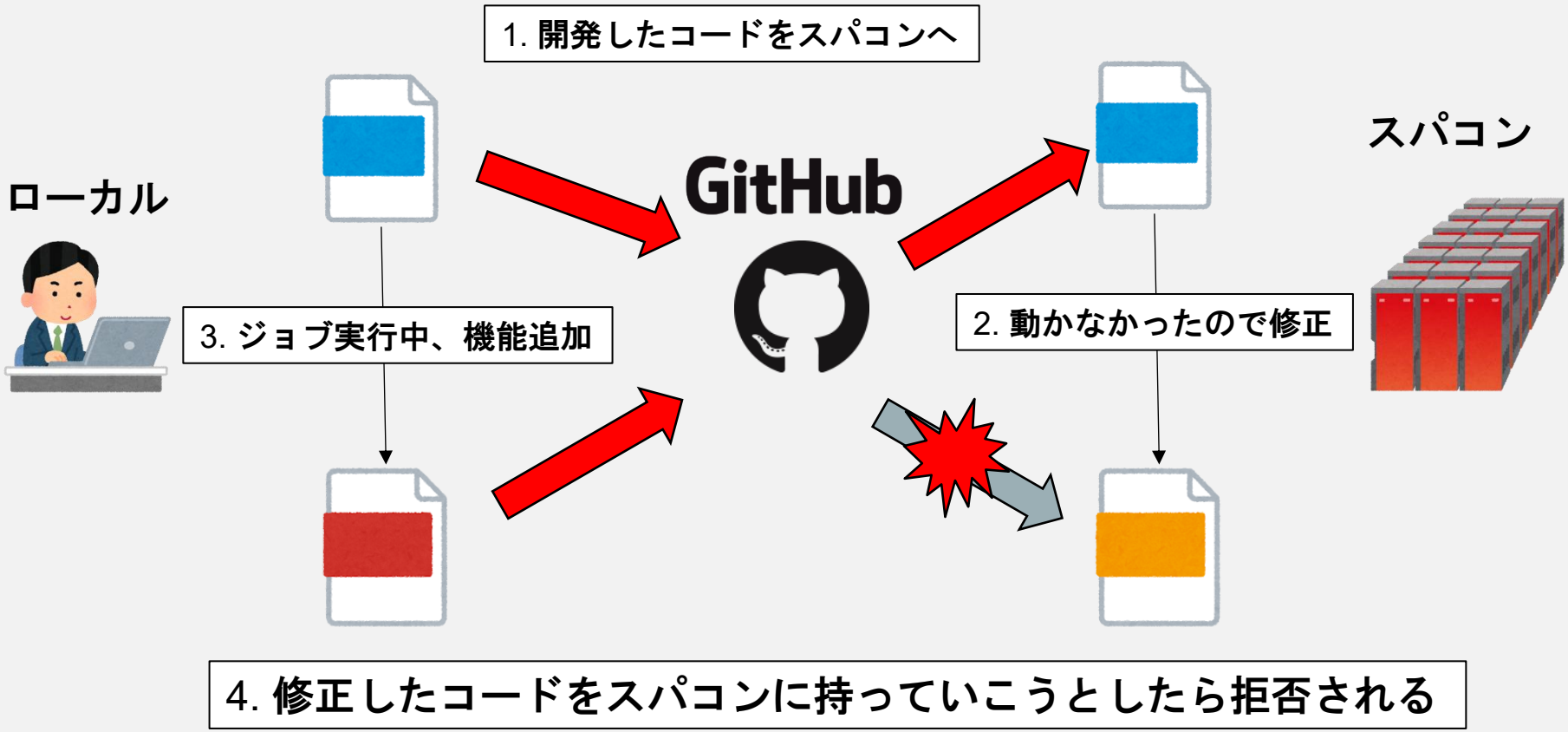

ローカル
スパコン
1. 開発したコードをスパコンへ
2. 動かなかったので修正
3. ジョブ実行中、機能追加
4. 修正したコードをスパコンへ
あ！上書きしちゃった！

バージョン管理してたら？



※ Overleafなどを使う人も多い

バージョン管理してたら？



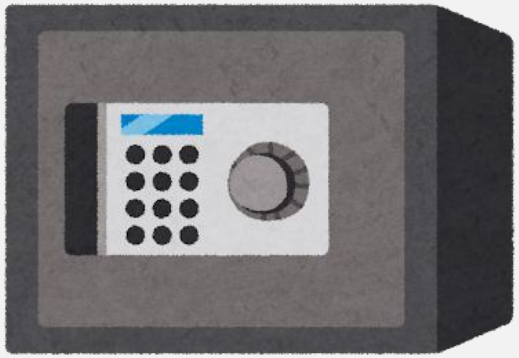
バージョン管理していると作業が失われない

バージョン管理システム

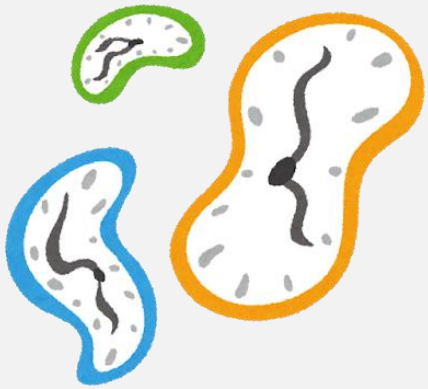
渡辺が
考える

バージョン管理システム導入の二大メリット

バックアップ



履歴保存



バックアップ

作業中のPCが壊れた！

間違って古いバージョンで上書き！
作業ミスで消しちゃった！

重要なファイルの消失により、大きな「手戻り」が発生する

バックアップ

バックアップは定期的にとる

バックアップ頻度＝データが飛んだ時の手戻りの時間
最低でも毎日バックアップすること

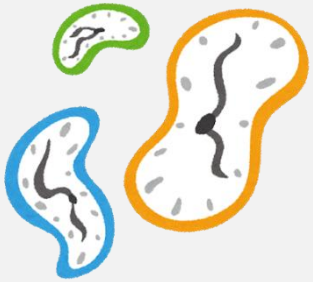
バックアップはリモートにとる

自分のPCの別フォルダにコピーするのはダメ
USBへのコピーも信用できない

➡ バージョン管理システムとして
Git/GitHubを使うと上記要件を
自動的に満たす

履歴保存

バージョン管理システムを使っていると…



- 任意の時点に状態を戻すことができる
- 任意の時点間の差分を確認できる

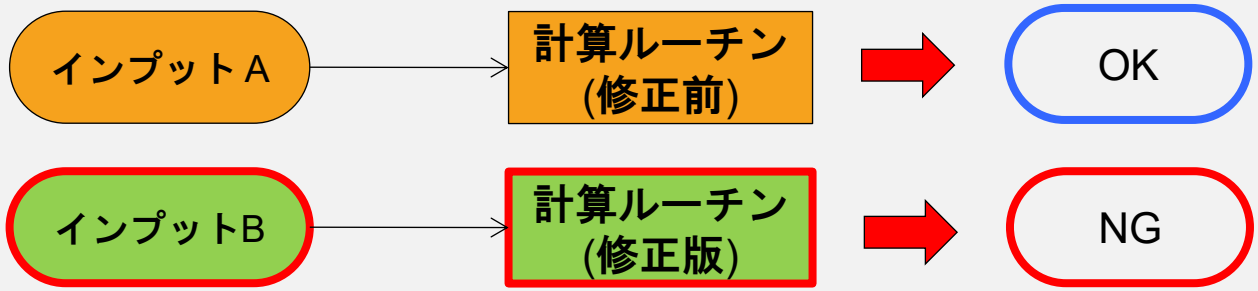
→ デバッグ時間の短縮

履歴保存とデバッグ

数値計算コードを開発中、

- メインカーネルを修正し
- 別のインプットを与えたら

計算に失敗した



その機能を追加したことによるバグ？
もともとあったバグがインプットにより顕在化？

履歴保存とデバッグ

バージョン管理システムを使っていないと…

ソースとにらめっこして
気合でデバッグ

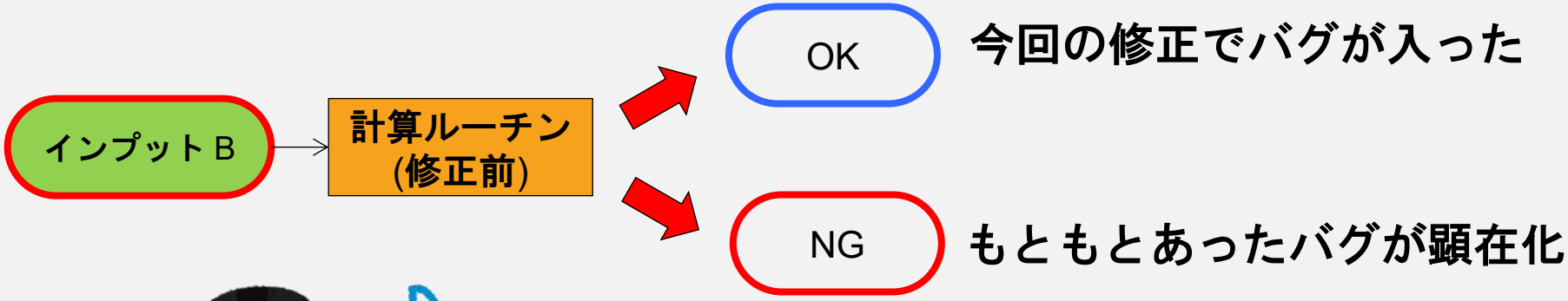
徹夜でなんとか
バグ発見

自分で入れたバグを自分でとっただけで、仕事はなんら進んでいない

履歴保存とデバッグ

バージョン管理システムを使っていれば…

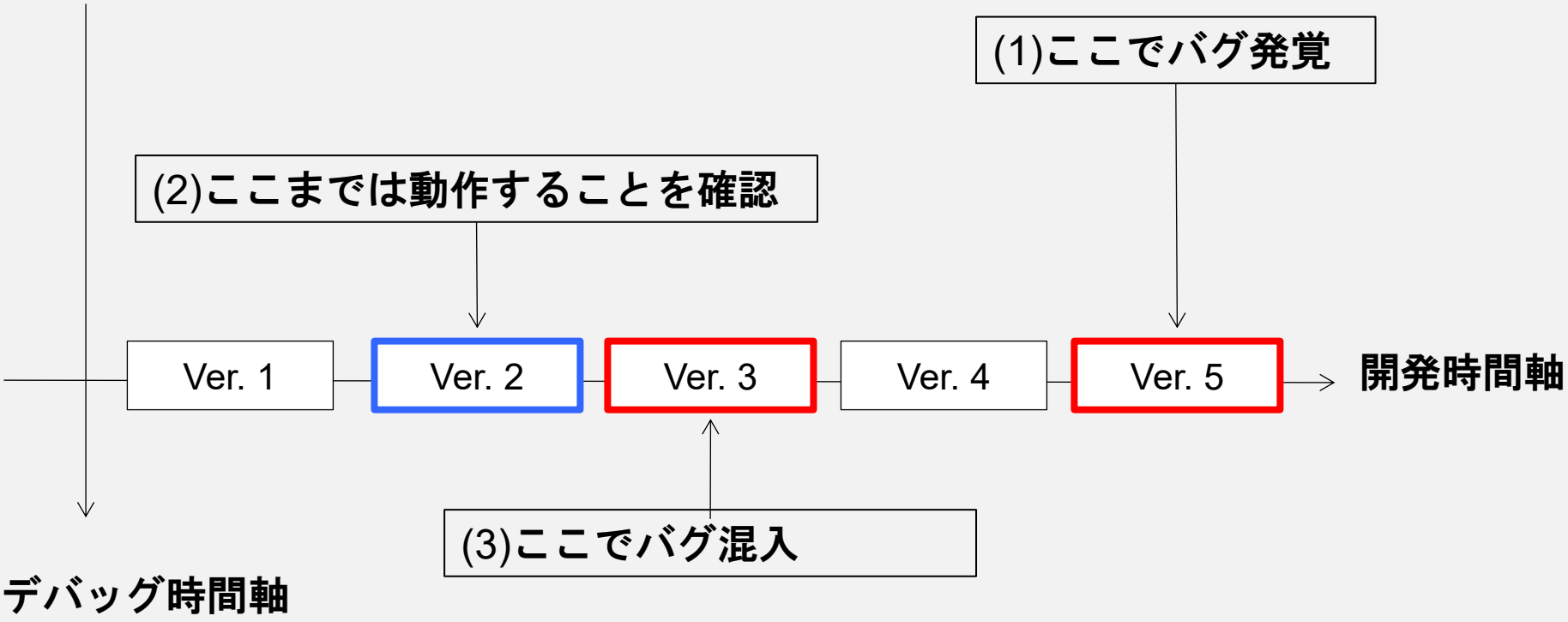
修正前のメインカーネルを取得し、Input Bを食わせる



問題の切り分けが容易

履歴保存とデバッグ

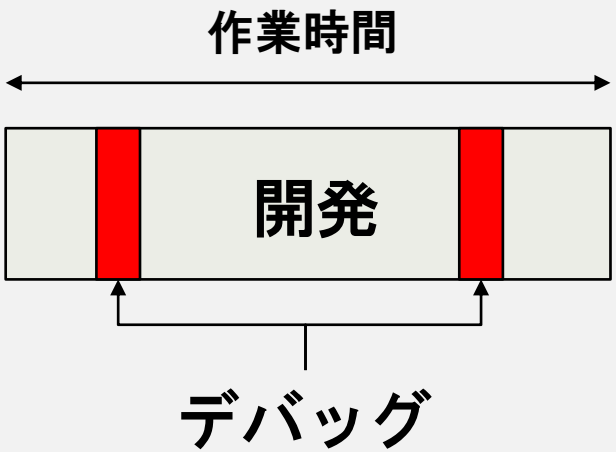
昔入れたバグほど、デバッグが困難に
(修正内容を忘れているから)



Ver. 2とVer. 3の差分を取れば、バグの原因がすぐにわかる

プログラミングのできる人、できない人

プログラミングが早い人は「デバッグ時間」が短い



左の人の方が「がんばっている」ように見えるが、
右の人の方が作業は進んでいる

見えないコスト

バージョン管理システムを使っていない人は
「バージョン管理システムを使っていれば節約できた時間」
を認識できない

開発	手戻り

PCが壊れてデータを失った人「自分は運が悪かった」

開発	デバッグ

デバッグに時間をかけているひと「自分はがんばっている」

VCSを使っていれば
この時間節約できたかも

バージョン管理システム

バージョン管理システムを使えばデバッグ時間が短くなるわけではない
「できる人」はGitが使えるから「できる」のではない

GitやGitHubはあくまでも「開発スタイル」を支援するツール
- Gitが「何を」実現するツールなのか
- 自分がGitで「何を」しようとしているのか

を理解することなしに開発効率は向上しない

まとめ

- バージョン管理システムとは、ドキュメントやソフトウェアのバージョンを管理するためのシステム
- 複数人開発で有用なツールだが、個人開発でも有用
  →三日前の自分は他人
- バージョン管理システムを使えば開発効率が上がる
  わけではない
  →開発スタイルを変えなくてはならない

Git/GitHubの使い方を学ぶことが目的ではない
ツールに流れる哲学を学ぶ